

Infralytix

Deployment & Operations Guide

Local setup, Docker & Docker Compose, CI/CD pipeline, production deployment across AWS/Azure/GCP, monitoring, backups, and the debug checklist for keeping Infralytix healthy in the wild.

Docker Compose

GitHub Actions CI/CD

Multi-Cloud Deploy

Monitoring & Backups

Table of Contents

Deployment & Operations Guide

1	Prerequisites & Local Setup	03
	Getting Infralytix running on your machine	
2	Docker & Docker Compose	04
	Containerizing every layer of the stack	
3	Environment Configuration	06
	Every variable the system needs, and why	
4	CI/CD Pipeline	07
	GitHub Actions — build, test, deploy	
5	Production Deployment	08
	AWS, Azure, and GCP deployment paths	
6	Monitoring & Logging	10
	Keeping an eye on Infralytix in production	
7	Backup & Disaster Recovery	11
	Protecting the database and generated artifacts	
8	Scaling Considerations	12
	Growing beyond a single instance	
9	Debug Checklist & Security Hardening	13
	Ship with confidence	

1

Prerequisites & Local Setup

Getting Infralytix running on your machine

Requirement	Notes
Docker & Docker Compose	Docker Engine 24+ / Docker Desktop with Compose v2
Python 3.12+	Only needed for running the backend outside Docker during development
Node.js 20+	For local frontend development outside Docker
A Gemini API key	Required for any AI-powered module to return real (non-mocked) output
A GitHub OAuth App	Client ID/secret for GitHub login and repository scanning

Quick Start

```
git clone https://github.com/<you>/infralytix.git
cd infralytix
cp .env.example .env          # fill in secrets — see Section 3
docker compose up --build     # starts frontend, backend, and Postgres

# App available at:
#   Frontend → http://localhost:5173
#   Backend  → http://localhost:8000/docs (FastAPI Swagger UI)
```

FIRST-RUN TIP

On first boot, run `docker compose exec backend alembic upgrade head` to apply database migrations before creating your first account.

2

Docker & Docker Compose

Containerizing every layer of the stack

docker-compose.yml (excerpt)

```
services:
  frontend:
    build: ./frontend
    ports: ["5173:5173"]
    environment:
      - VITE_API_URL=http://localhost:8000
    depends_on: [backend]

  backend:
    build: ./backend
    ports: ["8000:8000"]
    env_file: .env
    depends_on: [db]
    volumes:
      - ./uploads:/app/uploads
      - ./generated:/app/generated

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: infralytix
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: infralytix
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Backend Dockerfile (excerpt)

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

WHY A MULTI-SERVICE COMPOSE FILE, NOT ONE CONTAINER

Keeping frontend, backend, and database as separate services mirrors how Infralytix would actually be deployed in production (separately scalable tiers) — and it's exactly the kind of topology Deployment Intelligence itself generates for user projects, so the platform "eats its own dog food."

3

Environment Configuration

Every variable the system needs, and why

Variable	Purpose
DATABASE_URL	SQLAlchemy connection string (PostgreSQL in prod, SQLite in dev)
JWT_SECRET	Signing key for access/refresh tokens — must match across all backend replicas
GEMINI_API_KEY	Auth for all AI Agent calls
GITHUB_CLIENT_ID / GITHUB_CLIENT_SECRET	GitHub OAuth login & repository access
AWS_/AZURE_/GCP_ credentials	Read-only pricing & metrics access for Cloud Intelligence (never write-scope)
VITE_API_URL	Frontend → backend base URL
CORS_ALLOWED_ORIGINS	Whitelist of frontend origins allowed to call the API

NEVER COMMIT

`.env` is git-ignored by default. Cloud provider credentials granted to Infralytix should be scoped read-only / pricing-and-metrics-only — the platform never needs write access to a user's cloud account.

4

CI/CD Pipeline

GitHub Actions — build, test, deploy

```
name: infralytix-ci
on: [push, pull_request]

jobs:
  backend-tests:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16-alpine
        env: { POSTGRES_PASSWORD: test }
        ports: ["5432:5432"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install -r backend/requirements.txt
      - run: pytest backend/tests --maxfail=1

  frontend-build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: "20" }
      - run: npm ci --prefix frontend
      - run: npm run build --prefix frontend

  deploy:
    needs: [backend-tests, frontend-build]
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build & push images
        run: |
          docker build -t $REGISTRY/infralytix-backend:${{ github.sha }} backend/
          docker build -t $REGISTRY/infralytix-frontend:${{ github.sha }} frontend/
          docker push $REGISTRY/infralytix-backend:${{ github.sha }}
          docker push $REGISTRY/infralytix-frontend:${{ github.sha }}
      - name: Deploy
        run: ./scripts/deploy.sh ${{ github.sha }}
```

This is, fittingly, close to the exact shape of workflow that Infralytix's own Deployment Intelligence Agent generates for user projects.

5

Production Deployment

AWS, Azure, and GCP deployment paths

Cloud	Recommended Path	Notes
AWS	ECS Fargate (backend + frontend) + RDS PostgreSQL	No servers to patch; scales each service independently
Azure	Azure Container Apps + Azure Database for PostgreSQL	Built-in autoscaling on HTTP concurrency
GCP	Cloud Run (backend + frontend) + Cloud SQL for PostgreSQL	Scales to zero for low-traffic portfolio hosting — cost-efficient

Recommended for a Portfolio Deployment

For a single-developer portfolio deployment, **GCP Cloud Run** is the pragmatic default: it scales to zero when idle (near-zero cost between demo views), requires no cluster management, and deploys directly from the container images built in Section 4's CI/CD pipeline.

CUTOVER CHECKLIST

Run Alembic migrations against the production database → confirm all Section 3 environment variables are set in the cloud provider's secret manager → deploy backend, then frontend → smoke-test the four core flows in Document 4's UAT checklist.

6

Monitoring & Logging

Keeping an eye on Infralytix in production

Signal	How It's Collected
Application logs	Structured JSON logs shipped to the cloud provider's native log service (CloudWatch / Log Analytics / Cloud Logging)
Host & container metrics	The same <code>psutil</code> + Docker SDK collectors that power Infrastructure Intelligence, self-applied to Infralytix's own containers
Uptime	External synthetic check against <code>/health</code> every 60 seconds
AI Agent error rate	Tracked per-agent in <code>agent_tasks</code> (status column) for quick regression detection

FUTURE UPGRADE PATH

Prometheus + Grafana are marked "future" in the technology stack; when introduced, they replace the ad-hoc `psutil` polling with a proper metrics pipeline without changing what is measured — only how it's collected and visualized.

7

Backup & Disaster Recovery

Protecting the database and generated artifacts

Asset	Backup Method	Frequency / Retention
PostgreSQL database	Managed automated snapshots (RDS / Cloud SQL / Azure Database backups)	Daily, 7-day retention minimum
Generated artifacts (Dockerfiles, diagrams)	Regenerable on demand from source project data — treated as a cache, not a backup target	N/A
Uploaded repositories	Ephemeral by design — deleted after analysis completes; not retained long-term	Purged post-analysis

Recovery Procedure

1. Provision a fresh database instance from the latest snapshot.
2. Point `DATABASE_URL` at the restored instance and redeploy the backend.
3. Run `alembic upgrade head` to confirm schema currency.
4. Smoke-test login and one Intelligence Module end-to-end.

8

Scaling Considerations

Growing beyond a single instance

- Backend is stateless (JWT auth, no server-side sessions) — scale horizontally by increasing container replica count behind the platform's load balancer.
- Long-running Agent calls are the primary scaling bottleneck; the SDD's synchronous-call design is intentionally shaped so a Celery/Redis task queue can be dropped in later without an API contract change.
- PostgreSQL connection pooling (via SQLAlchemy's pool settings) should be tuned before adding backend replicas, to avoid exhausting the database's max connections.
- Static frontend assets should sit behind a CDN once traffic grows beyond demo-scale.

9

Debug Checklist & Security Hardening

Ship with confidence

Debug Checklist

Check	Where to Look
Backend won't start	<code>docker compose logs backend</code> — usually a missing env var or failed migration
Frontend can't reach API	Confirm <code>VITE_API_URL</code> and that <code>CORS_ALLOWED_ORIGINS</code> includes the frontend's origin
AI Agent calls failing	Verify <code>GEMINI_API_KEY</code> is set and not rate-limited; check <code>agent_tasks</code> table for the error payload
GitHub login fails	Confirm OAuth callback URL registered in the GitHub App matches the deployed domain exactly
Database connection errors	Check <code>DATABASE_URL</code> and that the db service/instance is reachable from the backend's network

Security Hardening Checklist

✓	Item
☐	HTTPS enforced at the load balancer; HTTP requests redirected, never served
☐	JWT secret and all API keys stored in the cloud provider's secret manager, not in plain environment files in production
☐	Cloud provider credentials scoped read-only (pricing/metrics), never write access
☐	CORS origin list contains only the production frontend domain
☐	Dependency and secret scans (Bandit / pip-audit) passing in CI before deploy
☐	Database backups verified restorable, not just "configured"

SUITE COMPLETE

This closes the five-document Infralytix suite: PRD → SRS → SDD → Test Plan → Deployment & Operations Guide — vision through to a live, monitored, recoverable production system.